Side-Channel Analysis of LESS

Shreyash Kumar

September 30, 2026

Abstract

This report presents a detailed technical analysis of potential side-channel vulnerabilities in the LESS (Linear Equivalence Signature Scheme) digital signature protocol. We focus on the `prepare_digest_input()` function and the embedded `lex_minimize()` routine, which performs constant-time lexical minimization of a column in the generator matrix G. We describe how a Flush+Reload attack can target the `lex_minimize()` code region to leak the `is_pivot_column` vector and discuss the implications for partial key recovery.

1 Introduction

LESS is a post-quantum digital signature scheme based on the Linear Equivalence Problem (LEP). Its security is predicated on the hardness of inverting a linear transformation over finite fields, with parameters typically chosen as:

- n (number of columns): 250–500,
- k (number of pivot columns): 120–250,
- q (field size): 127.

The scheme uses:

- A public generator matrix G.
- A private monomial matrix Q, which decomposes as

$$Q = P \cdot D,$$

  where P is a permutation matrix and D is a diagonal scaling matrix.
- The monomial matrix Q is represented in the scheme using two arrays:
  - $Q.u$, which stores the coefficients (scaling factors from D).
  - $Q.\pi$, which stores the permutation mapping from P.

2 Overview of Vulnerability

A side-channel vulnerability exists in the signing procedure, specifically within the `prepare_digest_input()` function. This function performs several operations:

1. It computes GQ via a monomial multiplication.
2. It transforms GQ into its Row-Reduced Echelon Form (RREF) and stores the resulting pivot pattern in a binary array called `is_pivot_column`.
3. Based on this pivot pattern, the function either:
   - Calls `lex_minimize()` on non-pivot columns, or
   - Records the corresponding permutation/scaling for pivot columns.

This control flow leads to distinct memory access patterns that can be exploited via a Flush+Reload attack.

3 Detailed Analysis of `lex_minimize()`

The `lex_minimize()` function performs constant-time lexical minimization of a column of G. Its goal is to normalize the column so that the first nonzero element is scaled to 1. This function is defined as follows:

```
/* constant time lexical minimization of a column of G, writes the
   result
 * as the desired column of a normalized_IS_t matrix */
void lex_minimize(normalized_IS_t *V,
                  POSITION_T dst_col_idx,
                  const generator_mat_t *const G,
                  const POSITION_T col_idx)
{
   POSITION_T first_nonzero_idx = 0;
   FQ_ELEM first_nonzero_val = 0;
   int nonzero_idx_found = 0;
   for (POSITION_T i = 0; i < K; i++) {
      first_nonzero_idx = ( i *
                            ( (G->values[i][col_idx] != 0) && !(
                               nonzero_idx_found) ) ) +
                          ( first_nonzero_idx *
                            ( (G->values[i][col_idx] == 0) ||
                               nonzero_idx_found ) );
      first_nonzero_val = ( G->values[i][col_idx] *
                            ( (G->values[i][col_idx] != 0) && !(
                               nonzero_idx_found) ) )+
                          ( first_nonzero_val *
                            ( (G->values[i][col_idx] == 0) ||
                               nonzero_idx_found ) );
      nonzero_idx_found = nonzero_idx_found || (G->values[i][col_idx]
         != 0);
   }
   FQ_ELEM inv_first = fq_inv(first_nonzero_val);
   for (uint32_t i = 0; i < K; i++) {
      V->values[i][dst_col_idx] = br_red(br_mul(G->values[i][col_idx],
         inv_first));
```

```
    }
} /* end lex_minimize */
```

Listing 1: `lex_minimize()` function

4 Flush+Reload Attack on `prepare_digest_input()`

4.1 Attack Methodology

The Flush+Reload attack proceeds as follows:

1. **Cache Flushing:** The attacker flushes the cache lines corresponding to the `lex_minimize()` function and, in particular, the memory region of `V->values[.][dst_col_idx]`.
2. **Execution Phase:** The victim executes `prepare_digest_input()`, where for each column:
   - If `is_pivot_column[col_idx] == 0`, `lex_minimize()` is called and the target memory is accessed.
   - Otherwise, the function writes directly to `Q_bar_IS`.
3. **Reload and Timing:** The attacker reloads the flushed memory lines. Fast access times indicate that the cache line was reloaded due to a call to `lex_minimize()`, while slow times indicate no access.

4.2 Extracted Information

The attacker thus recovers the `is_pivot_column` vector:

$$\texttt{is_pivot_column} = [1, 0, 1, 0, 1, \dots]$$

where a 1 denotes that the column was treated as a pivot (i.e., `lex_minimize()` was *not* called) and a 0 indicates a non-pivot column (with `lex_minimize()` executed). This binary vector provides partial information about the secret monomial matrix Q.

5 Mathematical Implications of the Leakage

5.1 Structure of Q

Recall that:

$$Q = P \cdot D,$$

where:

- P is the permutation matrix that reorders the columns of G.
- D is a diagonal matrix containing scaling factors from $\mathbb{F}_{127}$.

5.2 Leakage and Partial Key Recovery

The leaked `is_pivot_column` vector provides pivot-related information about the columns of G.Q. The obtained `is_pivot_column`, however, cannot be backtracked to the permutation part Q.π. The RREF operation ensures this, being a many-to-one function. The information can be used to find candidates for Q.π by generating all possible permutation matrices and multiplying them with G, followed by RREF calculation. The pivot pattern of thus obtained can be compared with the leaked pivot pattern. This reduces the search space for Q.

5.3 Parameter Impact and Computational Feasibility

Consider typical parameters:

- $n \approx 400$ columns, $k \approx 200$ pivots.
- The leakage produces a 400-bit vector with approximately 200 ones.

Even though the pivot pattern reduces the candidate permutations, the combined space of P and D remains computationally prohibitive for brute-force attacks in a post-quantum setting.
Even with the reduced candidate space from the pivot leakage:

- The remaining permutation candidates are still astronomical.
- The space for the diagonal scaling factors is 126^n; for $n \approx 400$, this is computationally infeasible.

Thus, while the leakage significantly reduces the search space, it is insufficient by itself to enable full key recovery under robust post-quantum parameter settings.

6 Conclusions and Future Work

- The function `prepare_digest_input()` leaks critical control flow and memory access patterns via the execution of `lex_minimize()`.
- A Flush+Reload attack targeting the `lex_minimize()` code and its associated data reveals the `is_pivot_column` vector, which in turn partially constrains the secret monomial matrix Q.
- The leakage provides partial information on the permutation P of Q, but the remaining unknowns (especially the diagonal scaling D) result in an enormous key space.
- Full key recovery remains computationally infeasible under typical post-quantum parameters, although the leakage could be combined with additional side channels to further reduce the search space.

Future work should investigate:

- Additional side-channel vectors that might leak information about the scaling factors.
- Implementation-specific vulnerabilities that could further constrain the key space.